“Fast” Neural Style Transfer in CVPR 2017

Yongcheng Jing

College of Computer Science and Technology

Zhejiang University

14/8/2017, Hangzhou

Content

- Introduction of Neural Style Transfer
  - “Slow” Neural Style Transfer and “Fast” Neural Style Transfer
- Introduction of three “Fast” Neural Style Transfer papers

Introduction

- What is Image Style Transfer?
  - Recombine the **content of a given photograph** and the **style of a well-known artwork**.
  - e.g.

Introduction

- Neural Style Transfer
  - Use **Convolutional Neural Network** to finish the task of image style transfer.
- Applications
  - Production Tools
  - Entertainment
  - Visualization & Presentation
  - Social Communication
    - e.g. “Prisma”, “Ostagram”, “In”

Introduction

- Neural Style Transfer
  - Use **Convolutional Neural Network** to finish the task of image style transfer.
- Applications
  - Production Tools
  - Entertainment
  - Visualization & Presentation
  - Social Communication
    - e.g. “Prisma”, “Ostagram”, “In”

Nine papers published in CVPR 2017 which study Neural Style Transfer

Also lots of related papers published in ICLR 2017, SIGGRAPH 2017, ACM MM 2017

Review of Neural Style Transfer

- Taxonomy of Neural Style Transfer Algorithms
  - **“Slow”** Neural Methods Based On **Image Optimization** (CVPR 2016 主战场)
  - **“Fast”** Neural Methods Based On **Model Optimization**
    - **Per-Style-Per-Model** “Fast” Neural Methods (CVPR 2016 主战场, CVPR 2017 主战场)
    - **Multiple-Style-Per-Model** “Fast” Neural Methods (CVPR 2017 主战场)
    - **Arbitrary-Style-Per-Model** “Fast” Neural Methods (ICCV 2017 主战场以及预计CVPR 2018 主战场)

Review of Neural Style Transfer

- Taxonomy of Neural Style Transfer Algorithms
  - **"Slow"** Neural Methods Based On **Image Optimization** (CVPR 2016 主战场)
  - **"Fast"** Neural Methods Based On **Model Optimization**
    - **Per-Style-Per-Model** "Fast" Neural Methods (CVPR 2016 主战场, CVPR 2017主战场)
    - **Multiple-Style-Per-Model** "Fast" Neural Methods (CVPR 2017 主战场)
    - **Arbitrary-Style-Per-Model** "Fast" Neural Methods (ICCV 2017 主战场以及预计CVPR 2018主战场)
  - Extensions
    - Color style transfer (https://github.com/LouieYang/deep-photo-styletransfer-tf, 150 stars in 2 days)
    - Typography style transfer
    - Visual attribute transfer

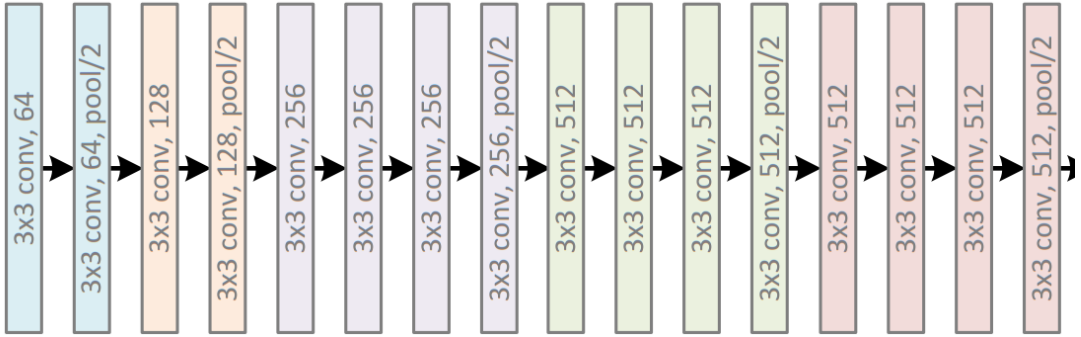
Paper collected at: https://github.com/ycjing/Neural-Style-Transfer-Papers

Review of Neural Style Transfer

- Overview of “Slow” algorithm:

Content Image

Style Image



Pre-trained VGG-19, fully-connected layers removed

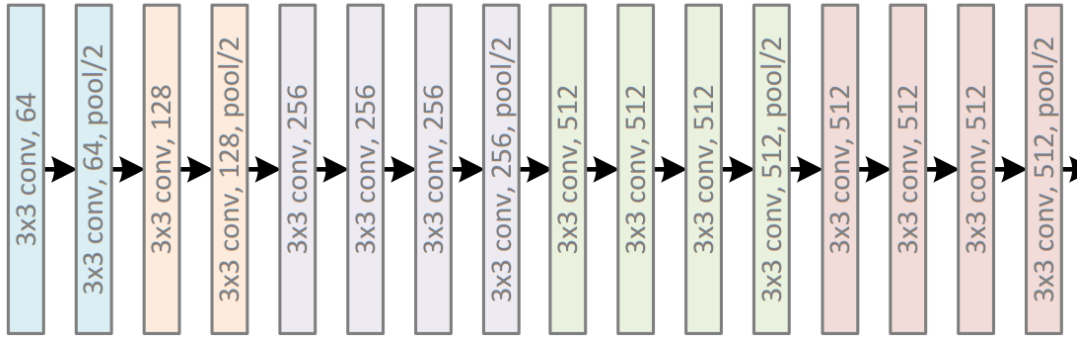
VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:

Content Image

Content features
512 x H x W

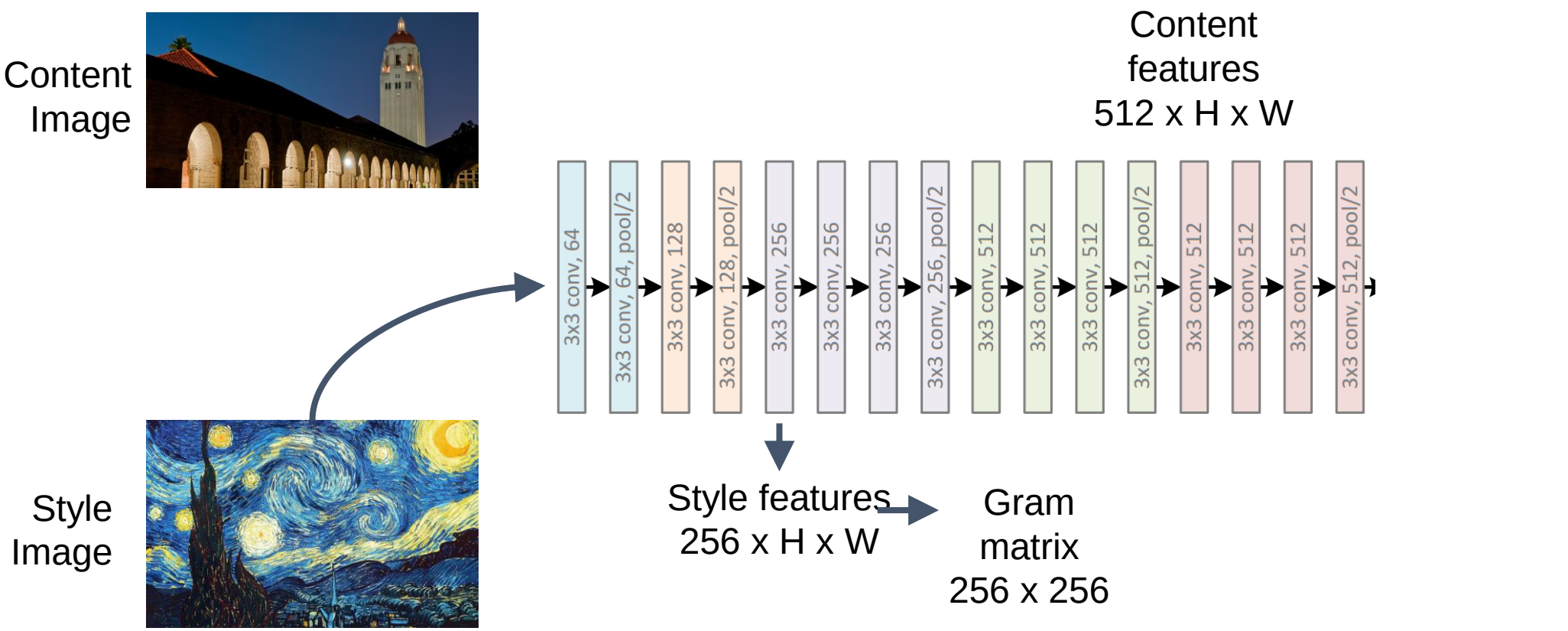


Style Image

VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:



Gram matrix计算方式为：
大小是 [256, H x W] 的feature map矩阵与其转置进行矩阵乘法

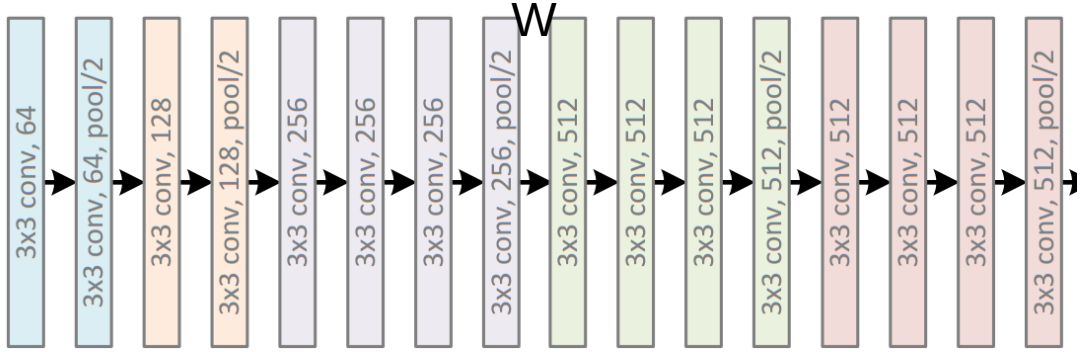
VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:

Content Image

Target features 512 x H x W



Style Image

Target gram matrix 256 x 256

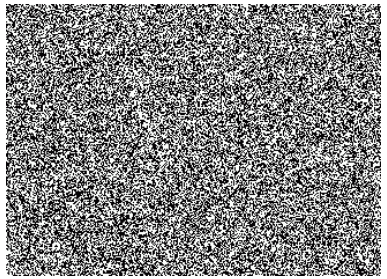
VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:

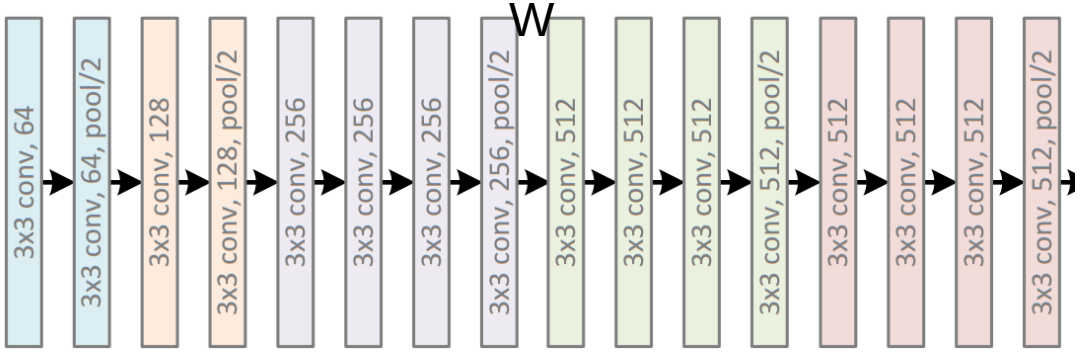
Content Image

Generated image



Style Image

Target features 512 x H x W

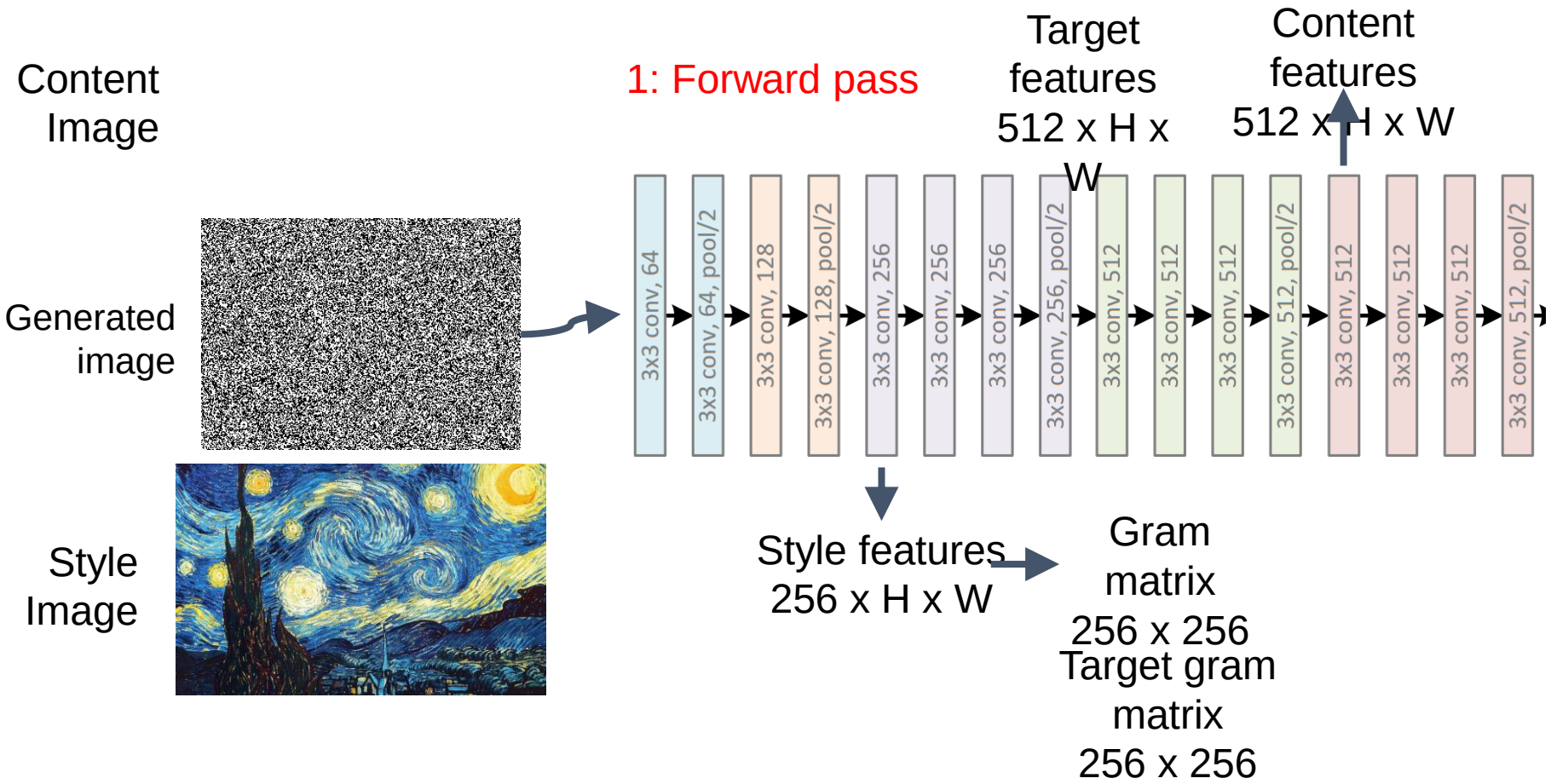


Target gram matrix 256 x 256

VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

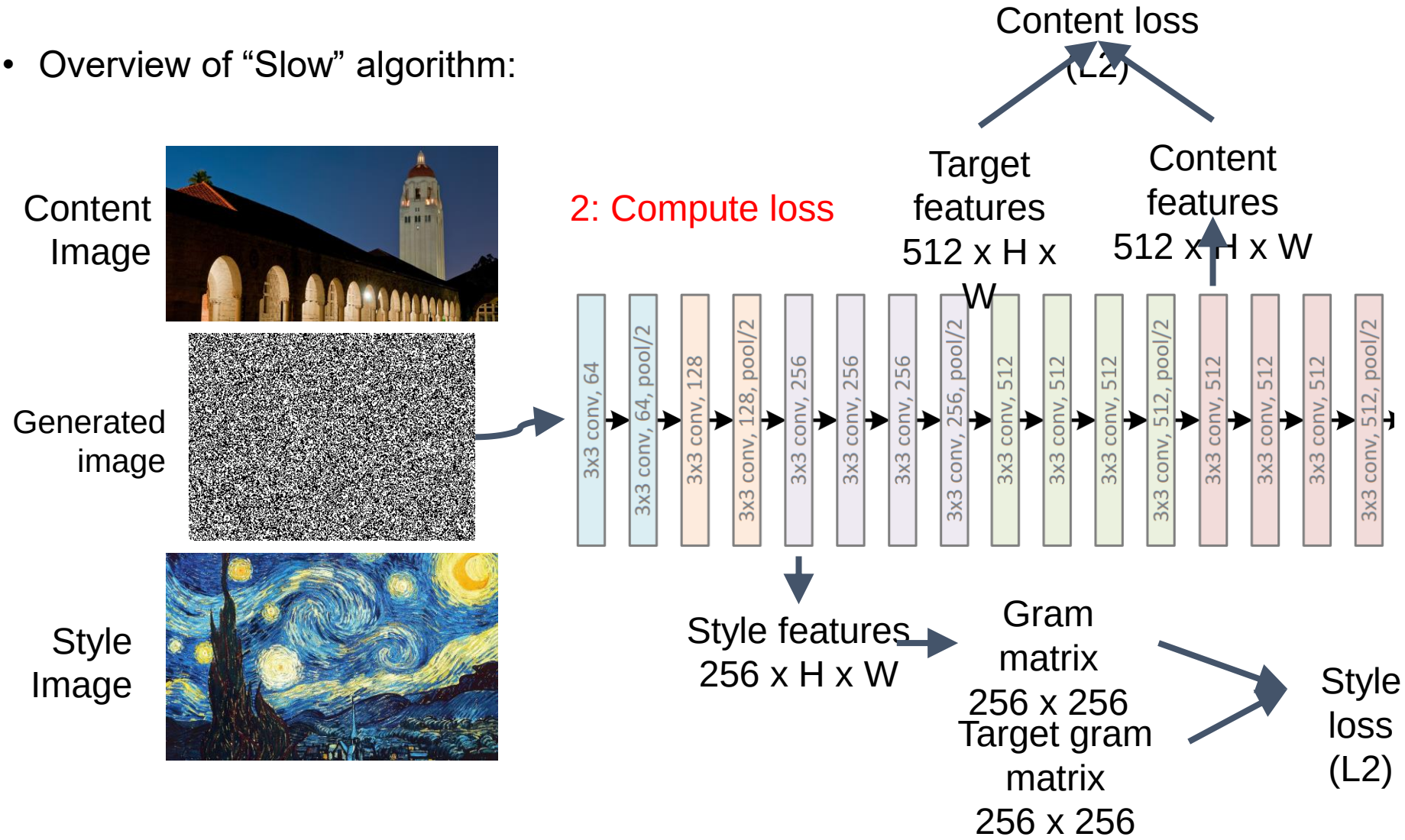
- Overview of “Slow” algorithm:



VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

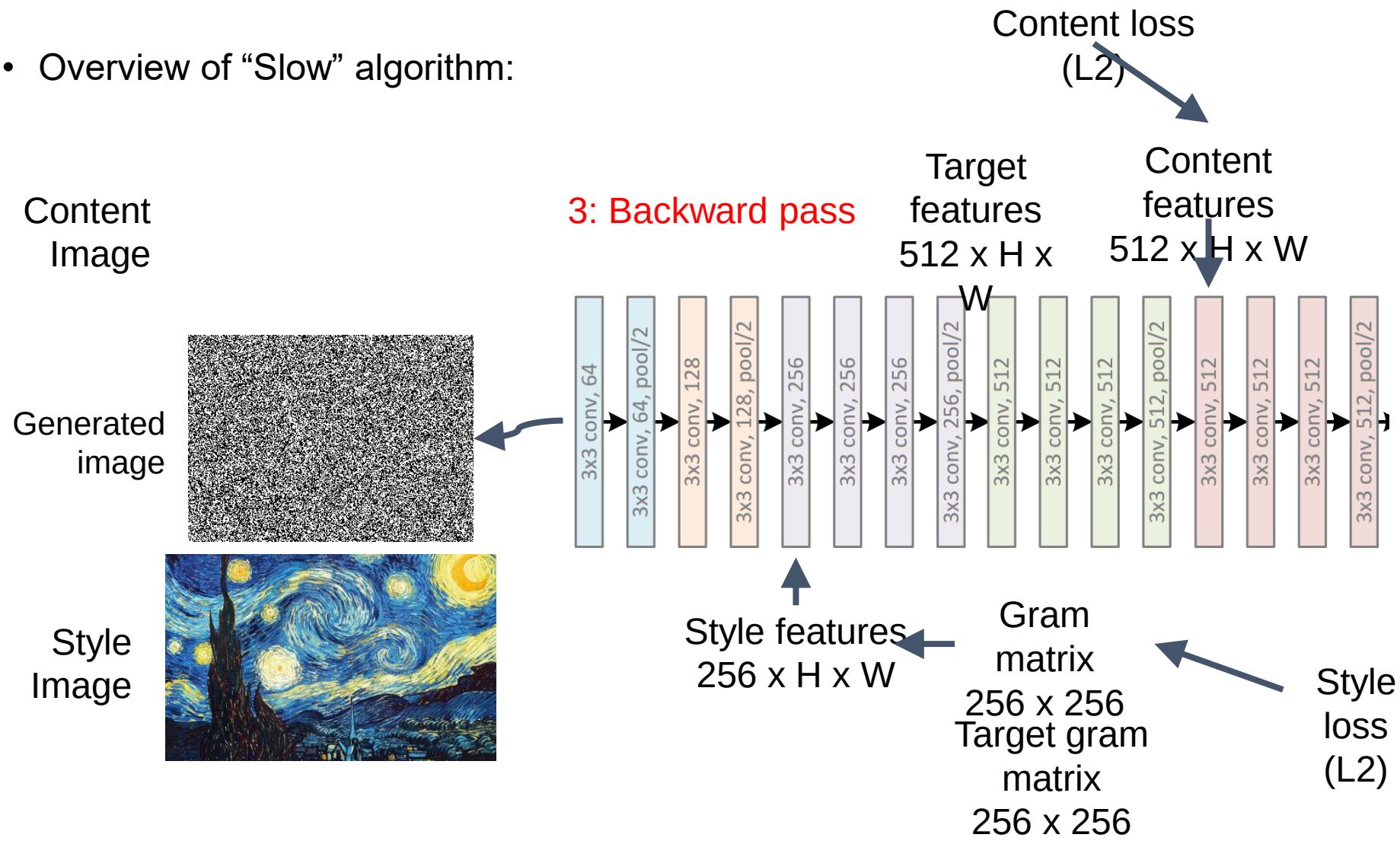
- Overview of “Slow” algorithm:



VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:



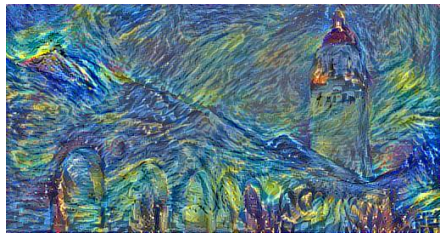
VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:

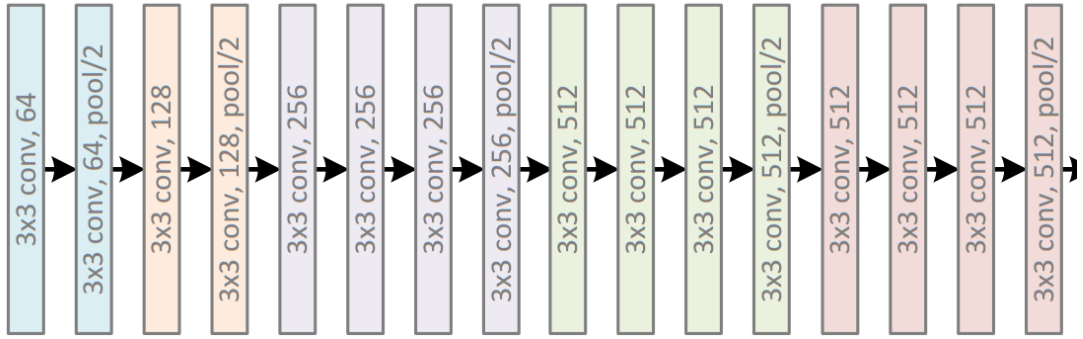
Content Image

Generated image



Style Image

4: Update image



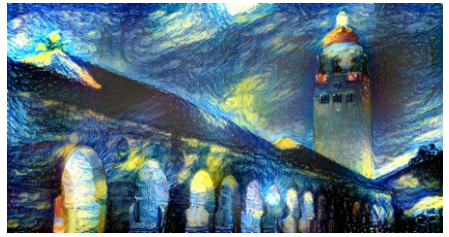
VGG figure credit: Kaiming He Other figures credit: Justin Johnson

Review of Neural Style Transfer

- Overview of “Slow” algorithm:

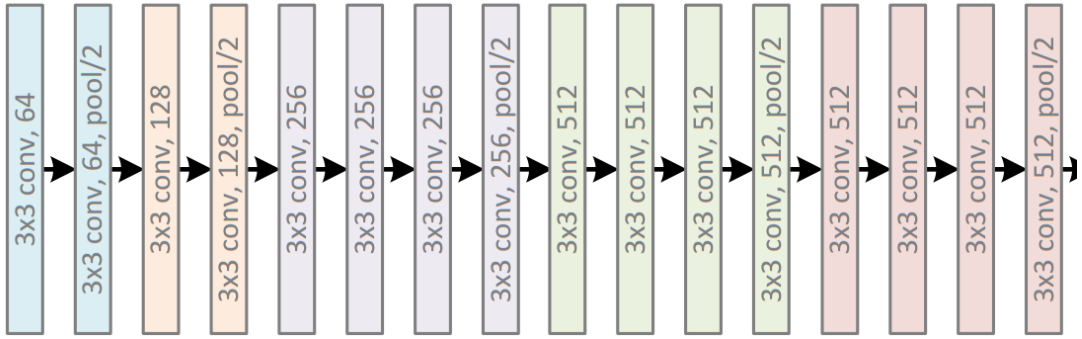
Content Image

Generated image



Style Image

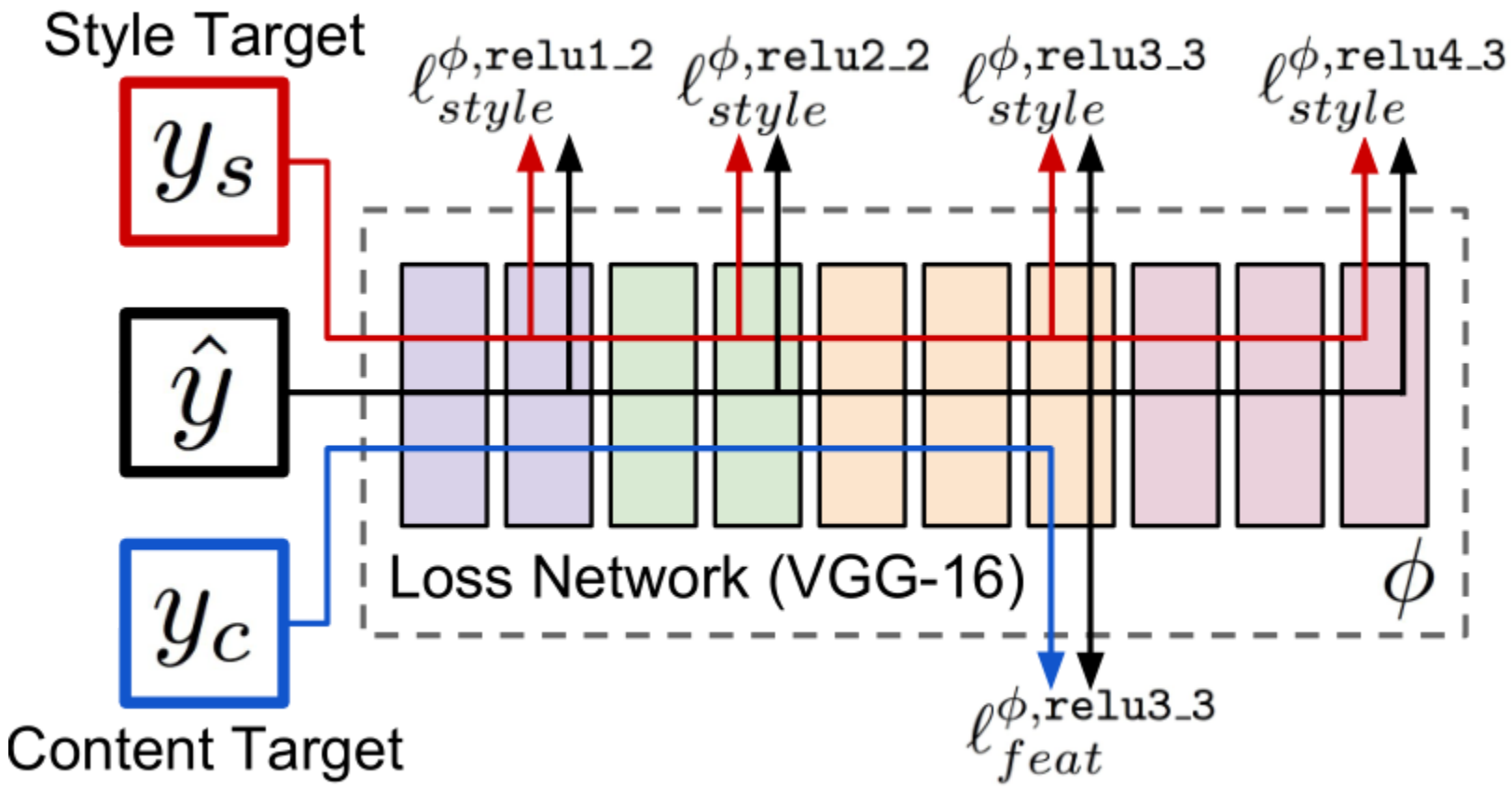
5: Repeat many times



VGG figure credit: Kaiming He Other figures credit: Justin Johnson

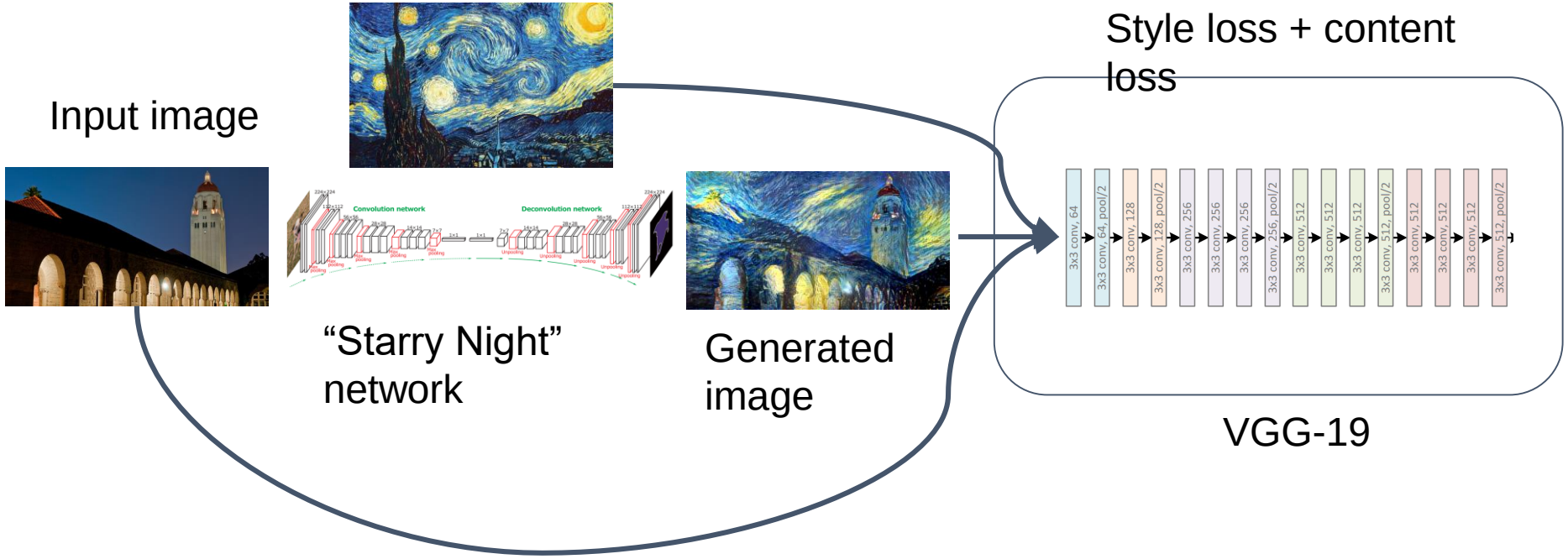
Review of Neural Style Transfer

- Loss function



Review of Neural Style Transfer

- Overview of per-style-per-model “Fast” algorithm:



Train a layer-specific style transfer model

Figures credit: Justin Johnson

Three Papers

- ① [Multimodal Transfer: A Hierarchical Deep Convolutional Neural Network for Fast Artistic Style Transfer]
  - Per-Style-Per-Model “Fast” algorithm
  - Solve the texture scale (or brush size problem) in previous “Fast” algorithm

High-resolution content | Style | Slow algorithm result | “Fast” algorithm result

(a) Content (b) Style (c) Results of Descriptive Method with Brush Size Control (d) Results of Generative Method without Brush Size Control

Three Papers

- ① [Multimodal Transfer: A Hierarchical Deep Convolutional Neural Network for Fast Artistic Style Transfer]
  - Per-Style-Per-Model “Fast” algorithm
  - Solve the texture scale (or brush stroke size problem) in previous “Fast” algorithm

High-resolution content | Style | Slow algorithm result | “Fast” algorithm result

(a) Content (b) Style (c) Results of Descriptive Method with Brush Size Control (d) Results of Generative Method without Brush Size Control

The reason is that 80000 training images are all resized to 256px to speed up the training process. If the test content is also 256px, the texture scale is good.

Three Papers

- ② **[StyleBank: An Explicit Representation for Neural Image Style Transfer]**
  - Multiple-Style-Per-Model
  - Support incremental learning
- ③ **[Diversified Texture Synthesis with Feed-forward Networks]**
  - Multiple-Style-Per-Model
  - Support incremental learning
  - Diversity loss
  - The authors provide1000-style model for research use.

Multimodal Transfer: A Hierarchical Deep Convolutional Neural Network for Fast Artistic Style Transfer

University of California, Santa Barbara and Adobe Research

Appeal

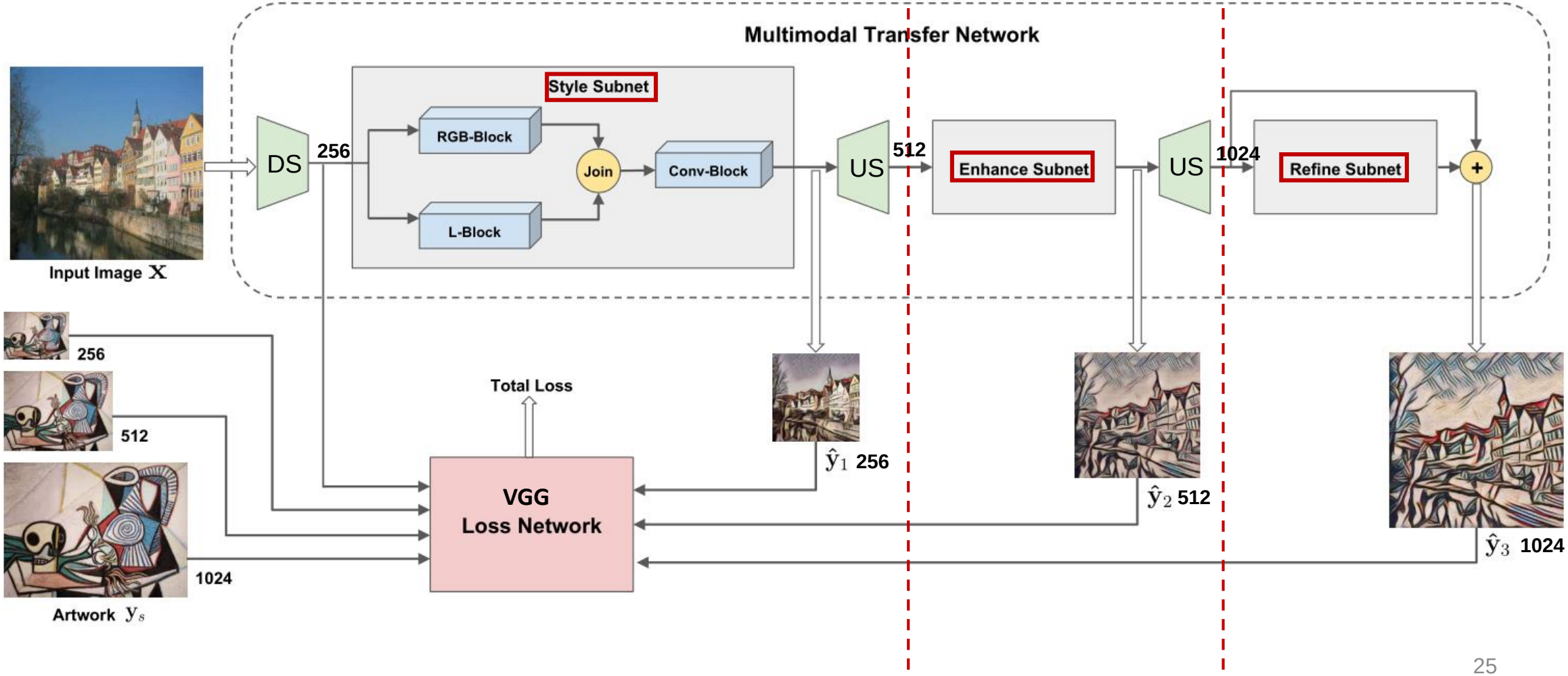
- Solve the small texture scale problem (or brush stroke size) in previous Per-Style-Per-Model algorithm (PSPM).

(a) Style | (b) Gatys *et al.* | (c) Johnson *et al.* | (d) Ulyanov *et al.* | (e) Ours | (f) Content

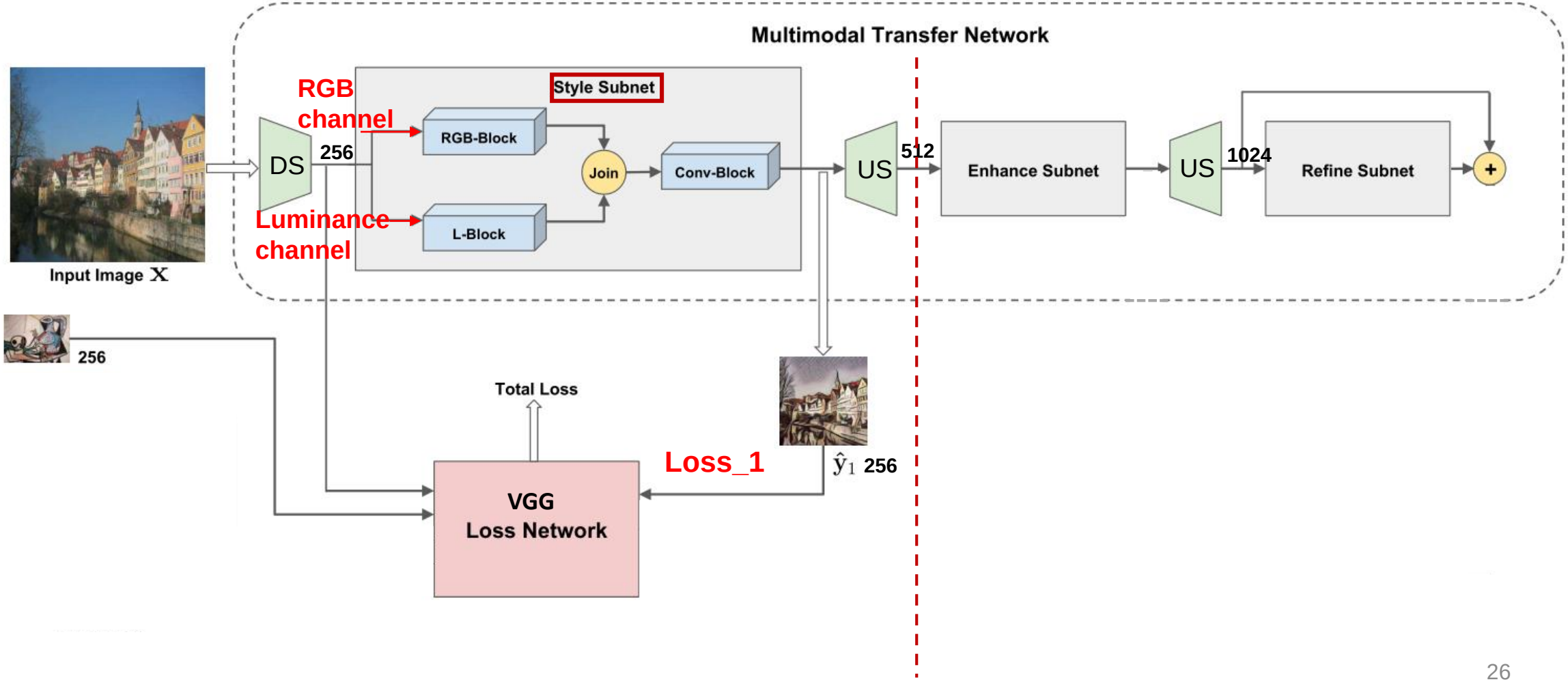
"Slow" algorithm | **PSPM algorithm #1** | **PSPM algorithm #2** | **This paper** | **1024px, high-resolution**

Method

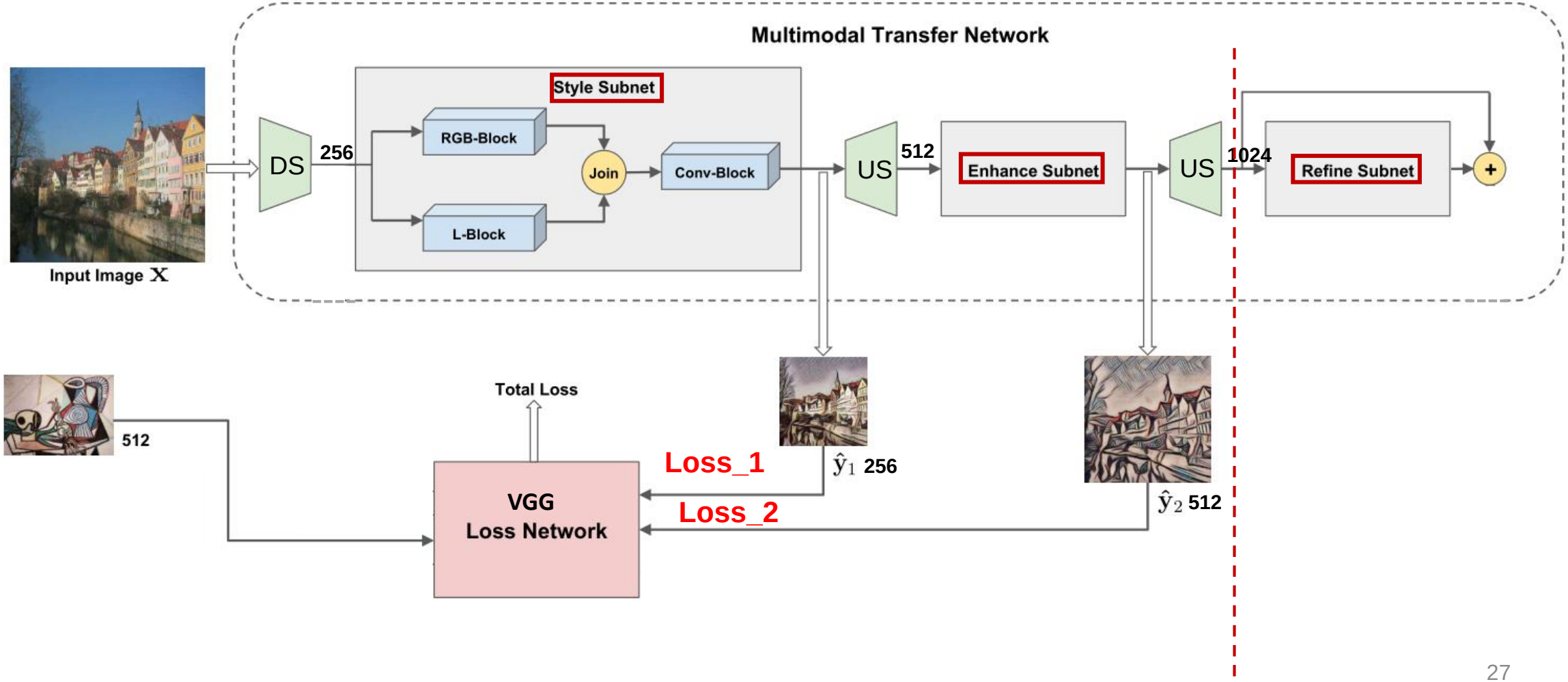
- **<u>Hierarchical Stylization (coarse-to-fine)</u>**
- **<u>VGG Loss function is the same as before, i.e. content loss and gram-based style loss.</u>**



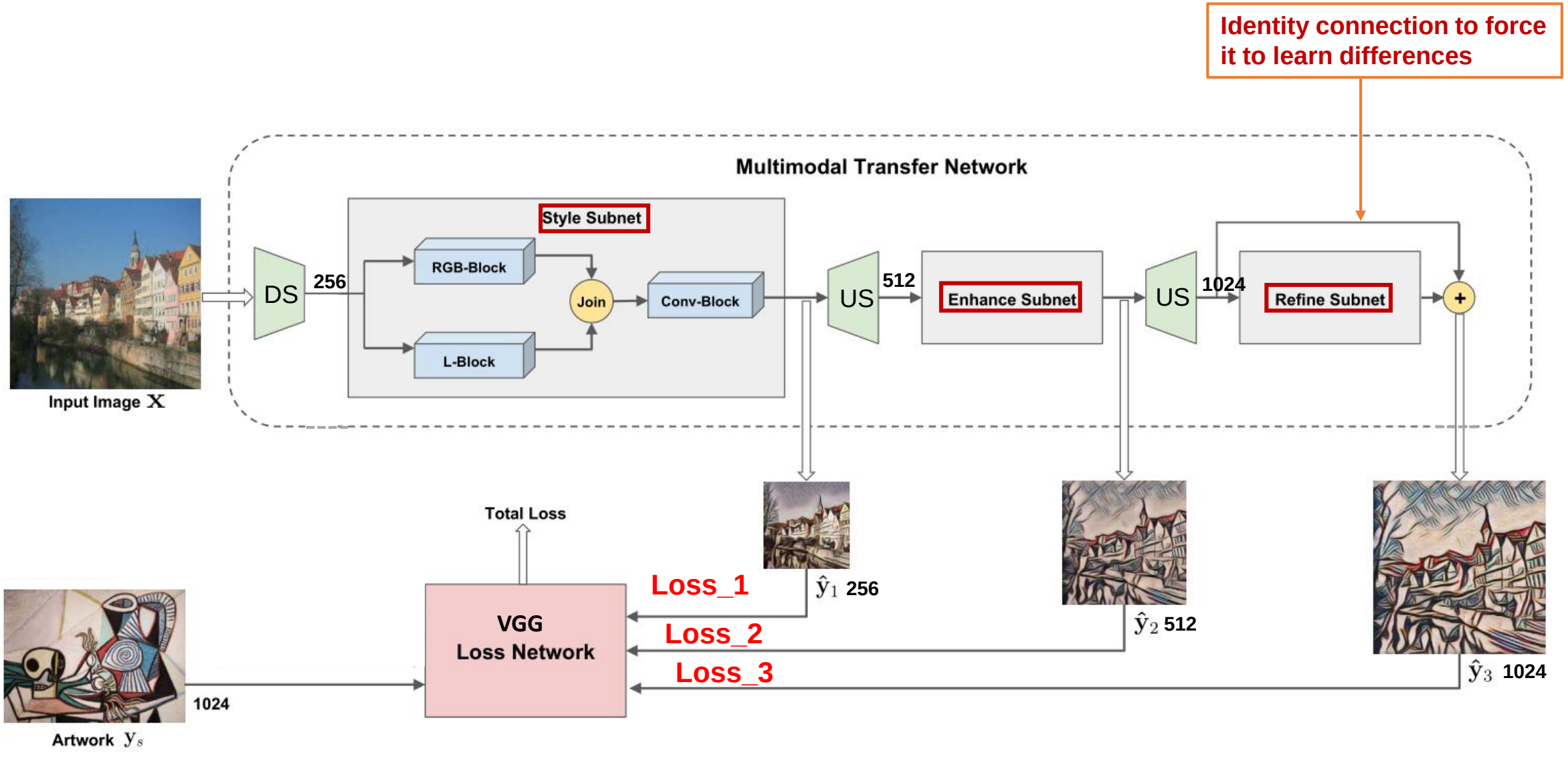
Method


Multimodal Transfer Network
Input Image X
DS
256
RGB channel
Luminance channel
Style Subnet
RGB-Block
L-Block
Join
Conv-Block
US
512
Enhance Subnet
US
1024
Refine Subnet
+
256
Total Loss
Loss_1
$\hat{y}_1$ 256
VGG
Loss Network

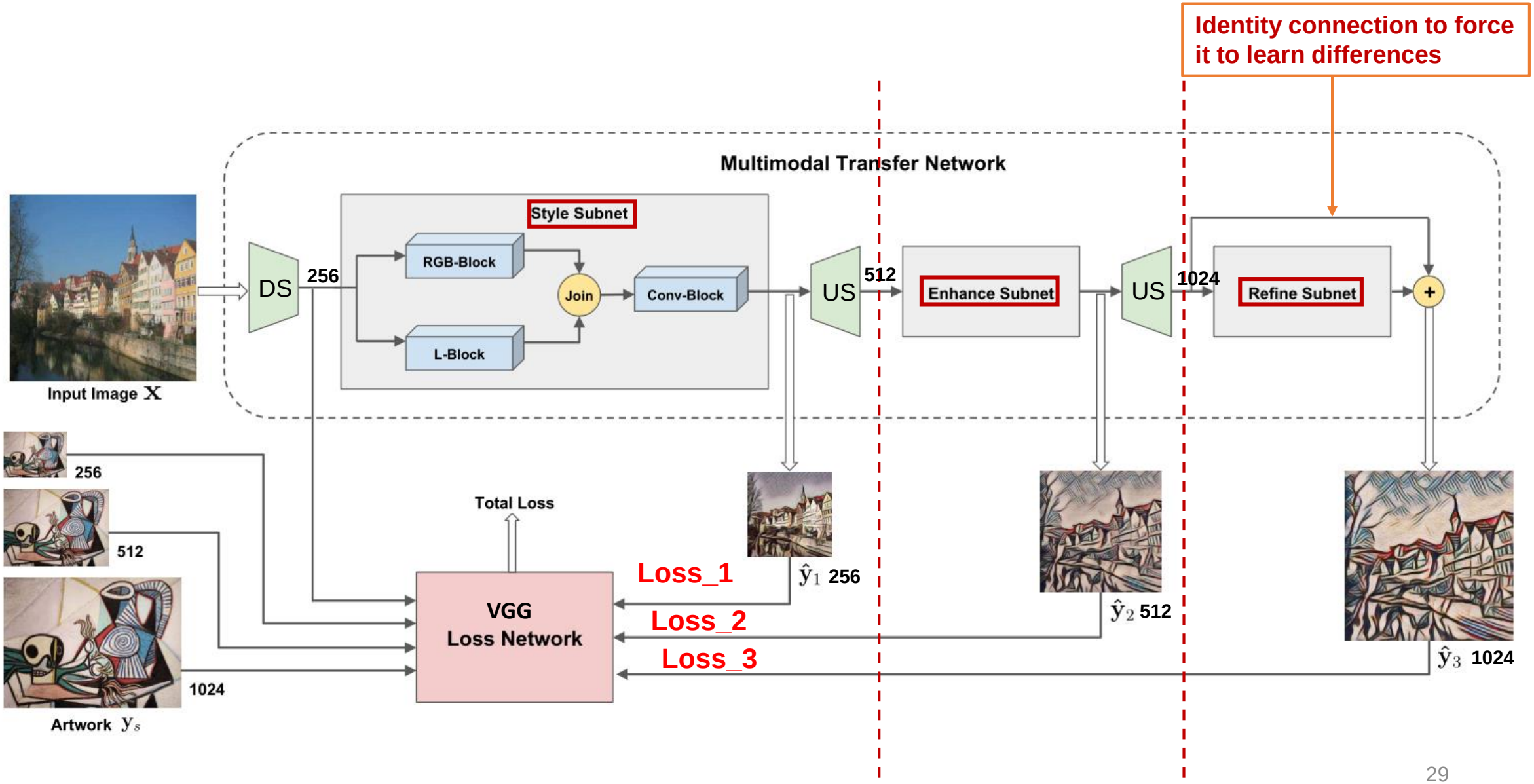
Method


Multimodal Transfer Network
Input Image X
DS
256
Style Subnet
RGB-Block
L-Block
Join
Conv-Block
US
512
Enhance Subnet
US
1024
Refine Subnet
+
512
Total Loss
VGG
Loss Network
Loss_1
Loss_2
ŷ1 256
ŷ2 512

Method

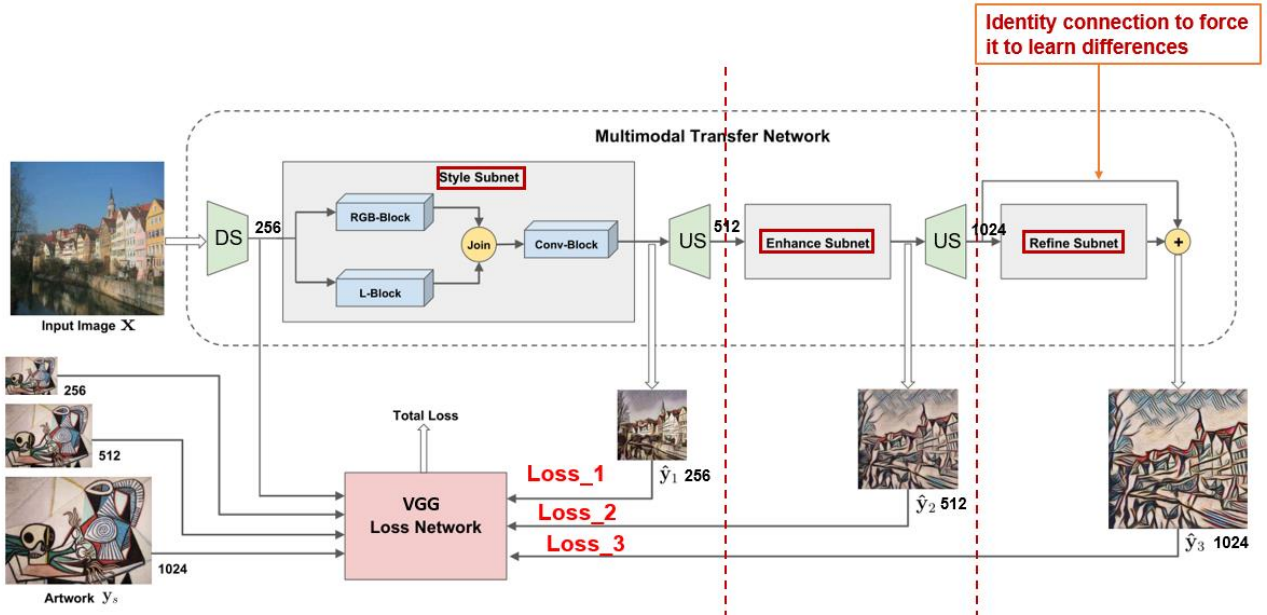

Identity connection to force it to learn differences
Multimodal Transfer Network
Input Image $\mathbf{X}$
DS
256
Style Subnet
RGB-Block
L-Block
Join
Conv-Block
US
512
Enhance Subnet
US
1024
Refine Subnet
+
$\hat{\mathbf{y}}_1$ 256
$\hat{\mathbf{y}}_2$ 512
$\hat{\mathbf{y}}_3$ 1024
Total Loss
VGG
Loss Network
Loss_1
Loss_2
Loss_3
1024
Artwork $\mathbf{y}_s$

Method


Identity connection to force it to learn differences
Multimodal Transfer Network
Input Image $\mathbf{X}$
DS
256
Style Subnet
RGB-Block
L-Block
Join
Conv-Block
US
512
Enhance Subnet
US
1024
Refine Subnet
+
256
512
1024
Artwork $\mathbf{y}_s$
Total Loss
VGG
Loss Network
Loss_1
Loss_2
Loss_3
$\hat{\mathbf{y}}_1$ 256
$\hat{\mathbf{y}}_2$ 512
$\hat{\mathbf{y}}_3$ 1024

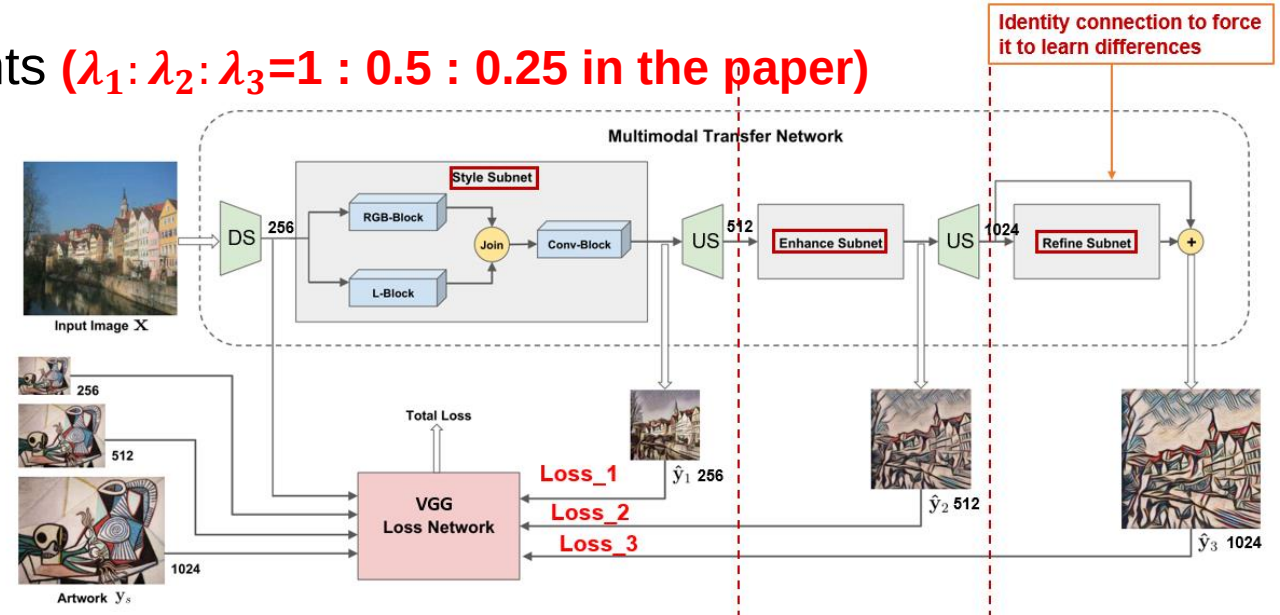
Method

- How to train?
  - The parameters of former subnets are updated to incorporate the **current and latter** stylization losses. → **(coarse-to-fine)**

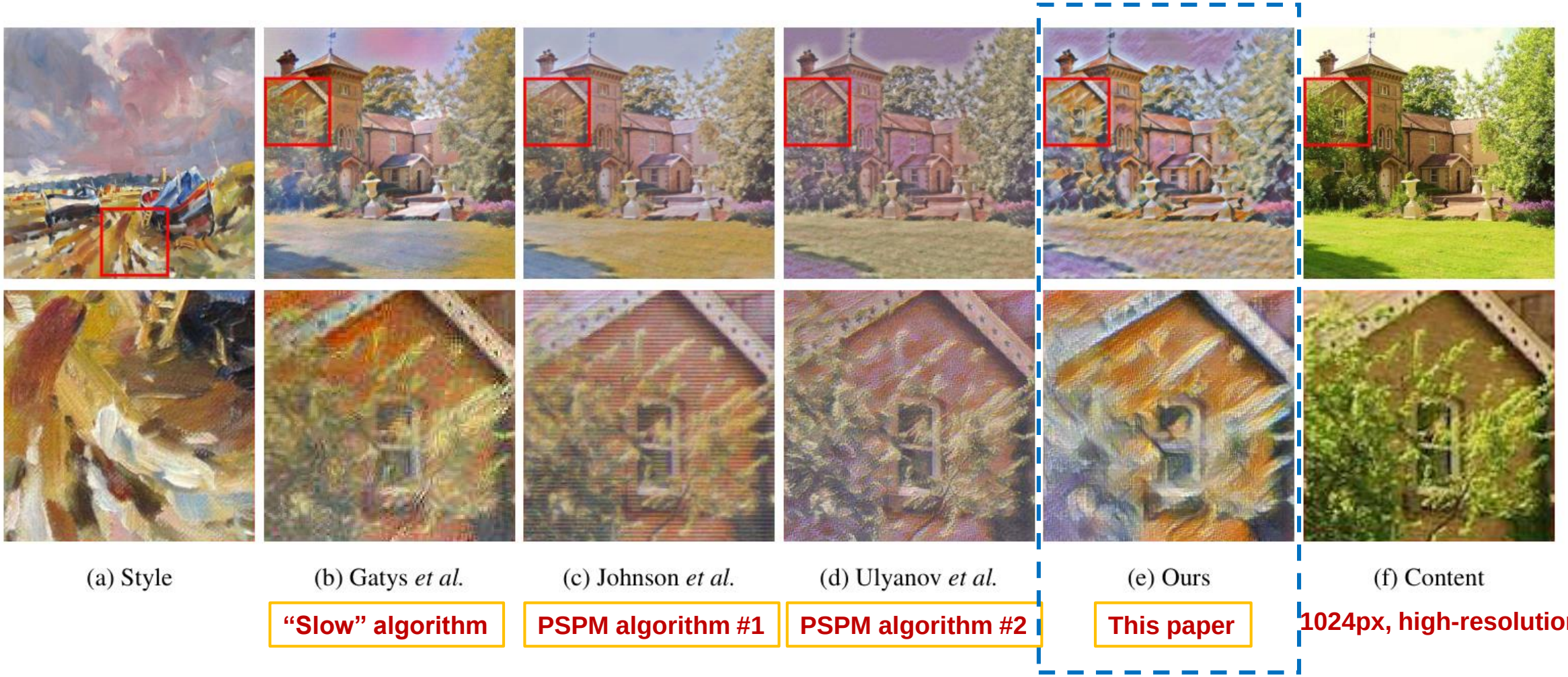


Method

- How to train?
  - The parameters of former subnets are updated to incorporate the **current and latter** stylization losses. → **(coarse-to-fine)**
  - i.e., in one iteration, the losses that each subnet optimizes is:
    - 1. [style subnet]: $\lambda_1 Loss_1 + \lambda_2 Loss_2 + \lambda_3 Loss_3$
    - 2. [enhance subnet]: $\lambda_2 Loss_2 + \lambda_3 Loss_3$
    - 3. [refine subnet]: $\lambda_3 Loss_3$
  - Latter subnet losses have **smaller** weights **($\lambda_1 : \lambda_2 : \lambda_3$=1 : 0.5 : 0.25 in the paper)**



Experimental Results



Conclusion

- **Coarse-to-fine** network design and training strategy
  - Subnetwork is trained to minimize the losses that are computed from the latter subnetwork outputs.

StyleBank: An Explicit Representation for Neural Image Style Transfer

University of Science and Technology of China and Microsoft Research

Appeal

- Multiple-Style-Per-Model
- Support incremental learning
  - Only need 8 minutes to add a new style into the model.

Analysis

- Key points to be considered for Multiple-Style-Per-Model:
  - 1. Choice of **signal** for each style: different style-specific filter bank
  - 2. **Scability:** One network to learn thousands of styles may not be a feasible solution.
  - 3. **Incremental (Online) learning** for new styles: train new filter bank

Analysis

- Inspiration for this paper:
  - For each content-style pair, actually the **target content is always fixed** for each style.
  - Therefore, in previous **Per-Style-Per-Model** method, it is **redundant** to train a network **both for content and style.** There may be something shared between different models.
  - Can we use **separate networks** to extract content representation and style representation which are **independent** of each other?

Method

- Network architecture

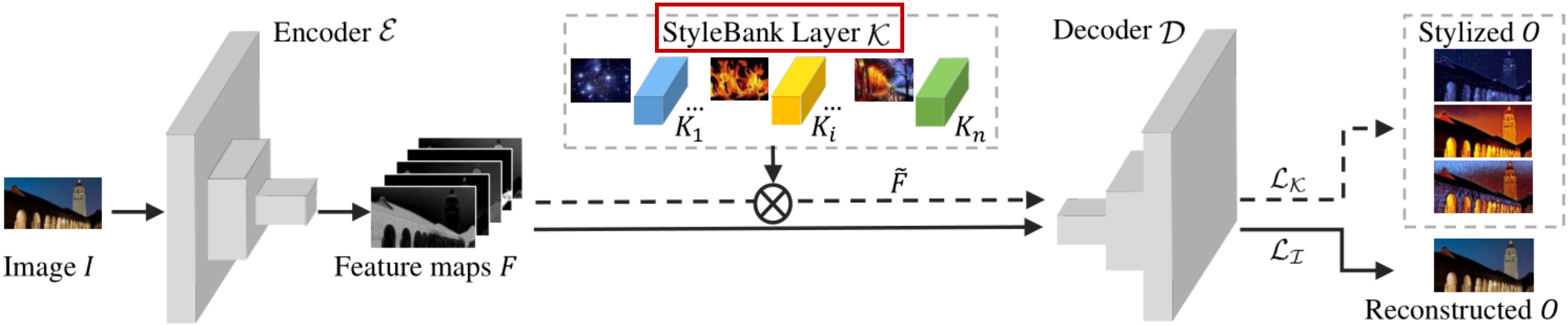


Figure 1. Our network architecture consists of three modules: image encoder $\mathcal{E}$, StyleBank layer $\mathcal{K}$ and image decoder $\mathcal{D}$

Method

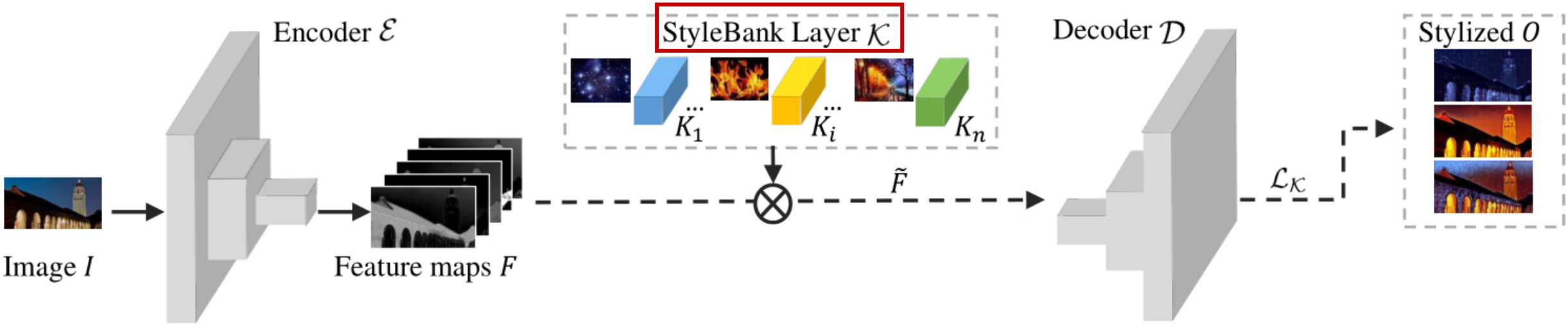
- Network architecture

Firstly, train **Auto-encoder** to learn **content representation**.
The objective is $\boldsymbol{O == I}$
Content Loss is the same as previous Neural Style algorithm.

Method

- Network architecture

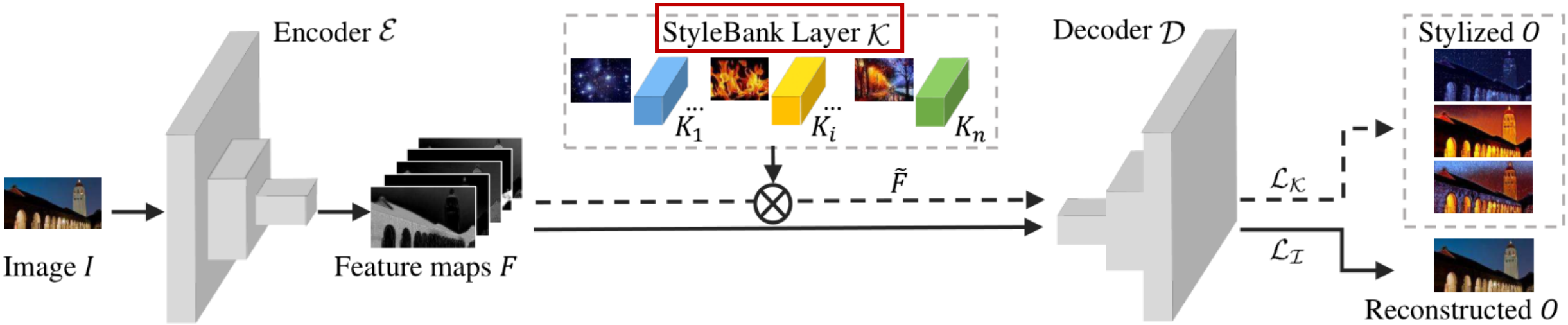
n is # of styles
Use StyleBank layer to **add style elements** into the content



Method

- Network architecture

n is # of styles
Use StyleBank layer to **add style elements** into the content

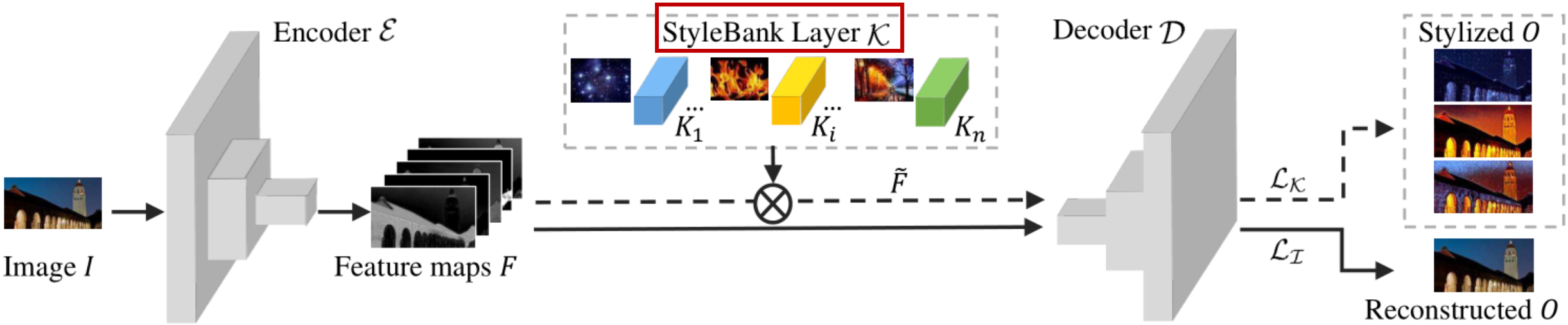


- Style loss is also the same as before, gram-based loss
- The training procedure for two branches is **inspired by GAN**:
  - For **T+1** iterations,
    - train **T** iterations on **branch L_K (虚线)**
    - train **1** iteration for **branch L_K (实线)**

Method

- Network architecture

n is # of styles
Use StyleBank layer to **add style elements** into the content

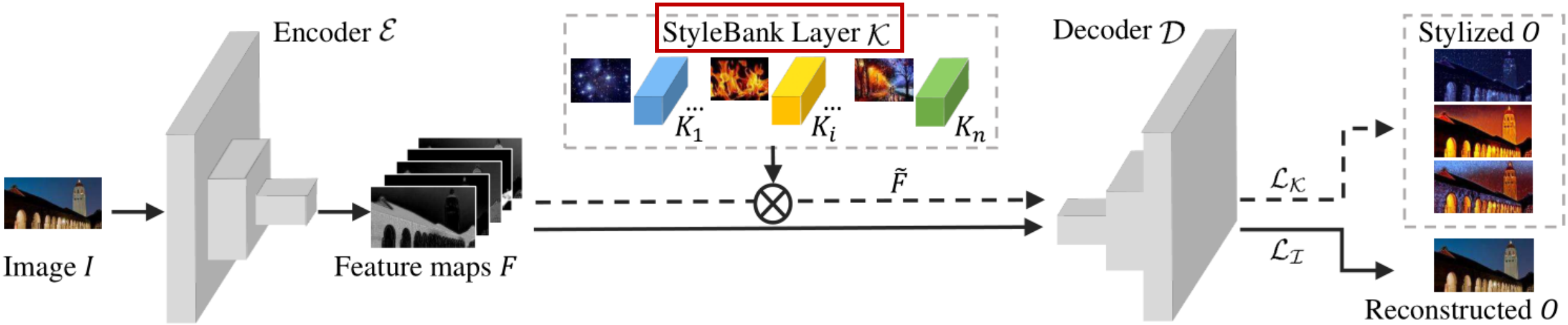


- Incremental learning:
  - **Fix auto-encoder** and only **train a new filter bank** in StyleBank Layer
  - 8 minutes to train for a new style

Method

- Network architecture

n is # of styles
Use StyleBank layer to **add style elements** into the content



- Key points to be considered for Multiple-Style-Per-Model:
  - 1. Choice of **signal** for each style: different style-specific filter bank
  - 2. **Scability:** One network to learn thousands of styles may not be a feasible solution.
  - 3. **Incremental (Online) learning** for new styles: train new filter bank

Experimental Results

Figure 11. Comparison with optimization-based method [12].

Conclusion

- **GAN-like** training strategy
  - Fixed one branch and train the other

GAN

Diversified Texture Synthesis with Feed-forward Networks

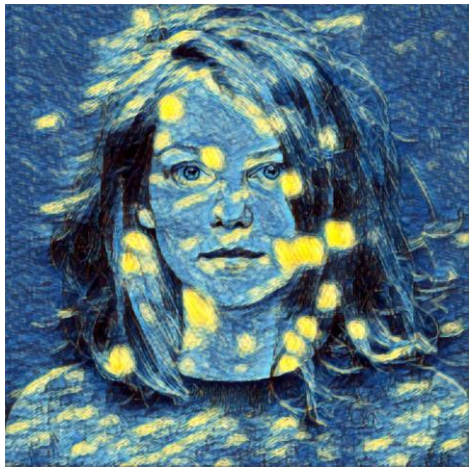
University of California, Merced and Adobe Research

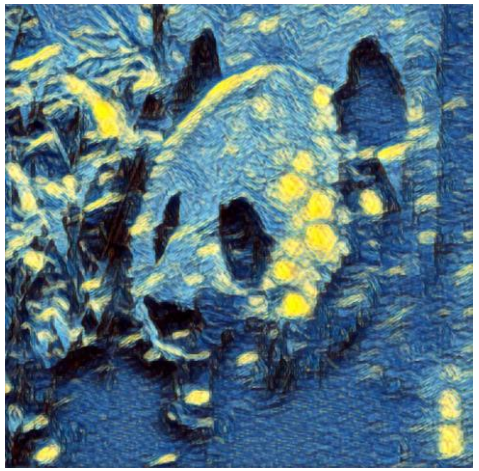
Appeal

- Multiple-Style-Per-Model
- Support incremental learning
- **Diversity loss**

Appeal

- Multiple-Style-Per-Model
- Support incremental learning
- **Diversity loss**





No diversity loss:

➢ Overfitting to a particular instance, repetitive patterns

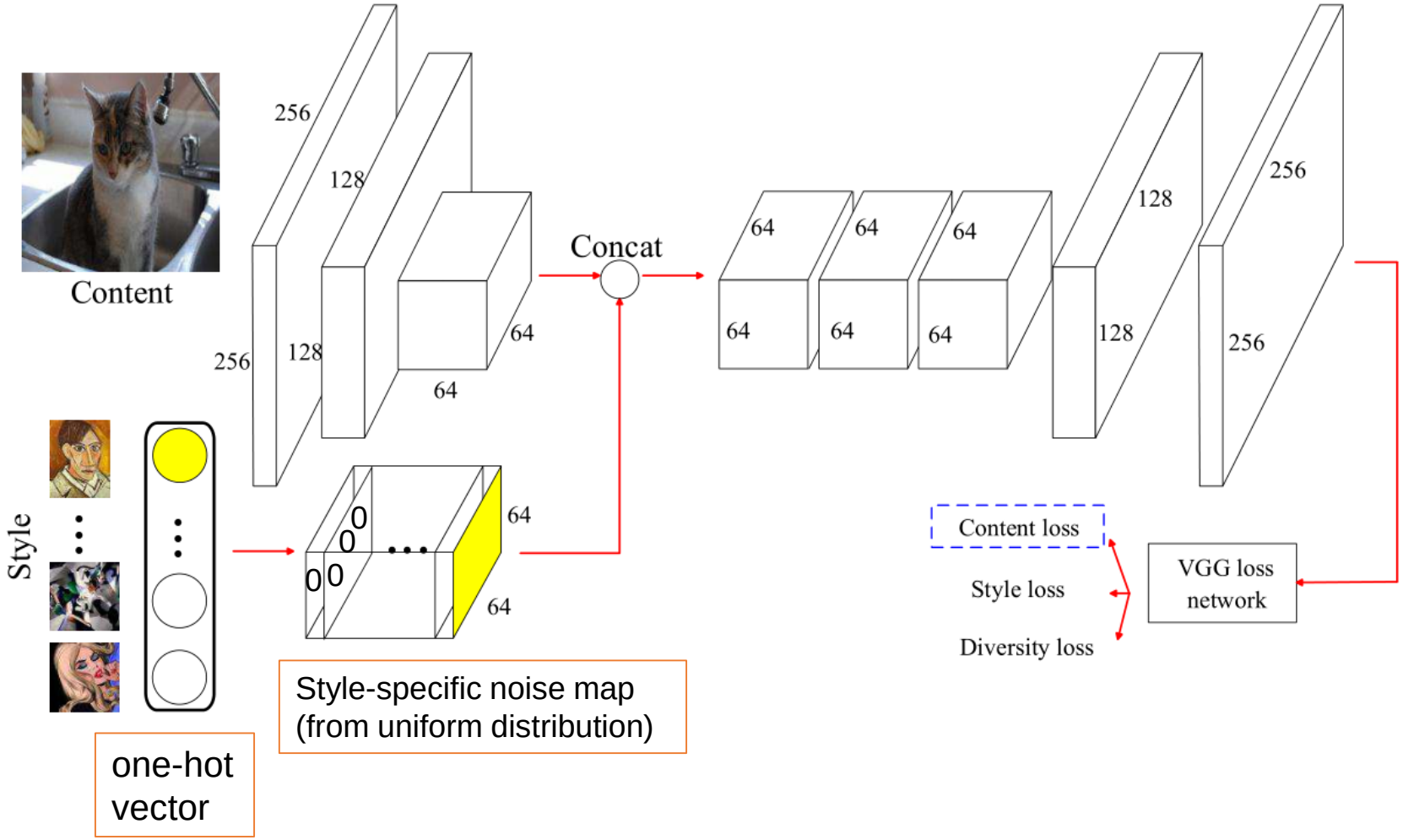
Analysis

- Key points to be considered for Multiple-Style-Per-Model:
  - 1. Choice of **signal** for each style: style-specific noise
  - 2. **Scability:** One network to learn thousands of styles may not be a feasible solution.

    (it actually works in this paper, the author provides 1000-style model; if larger, I doubt it)
  - 3. **Incremental (Online) learning** for new styles

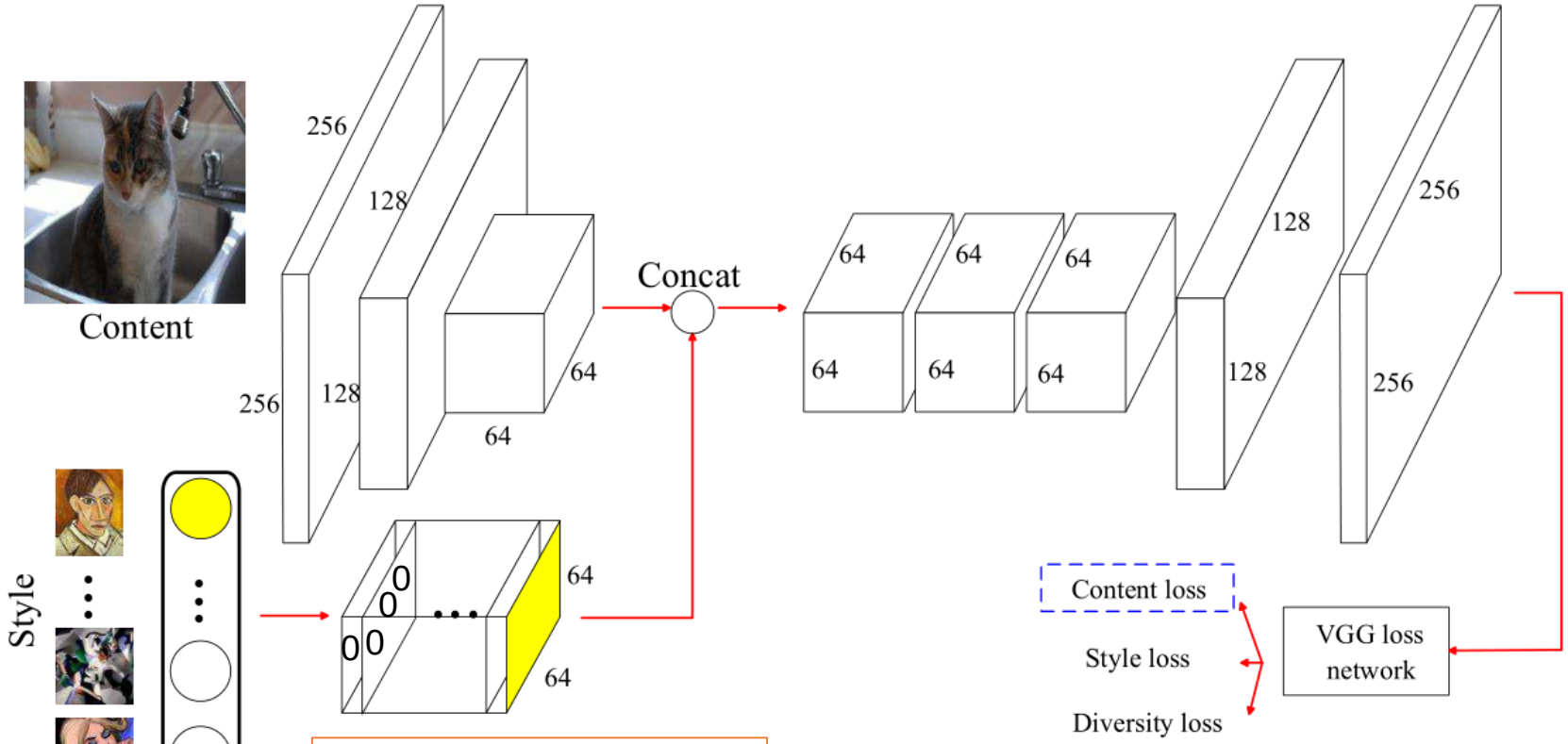
Method

- Network architecture



Method

- Network architecture

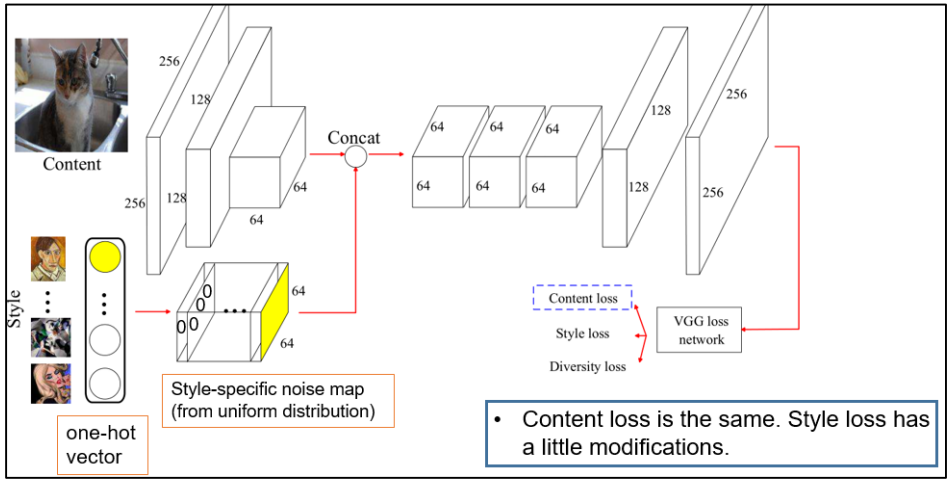


one-hot vector

Style-specific noise map (from uniform distribution)

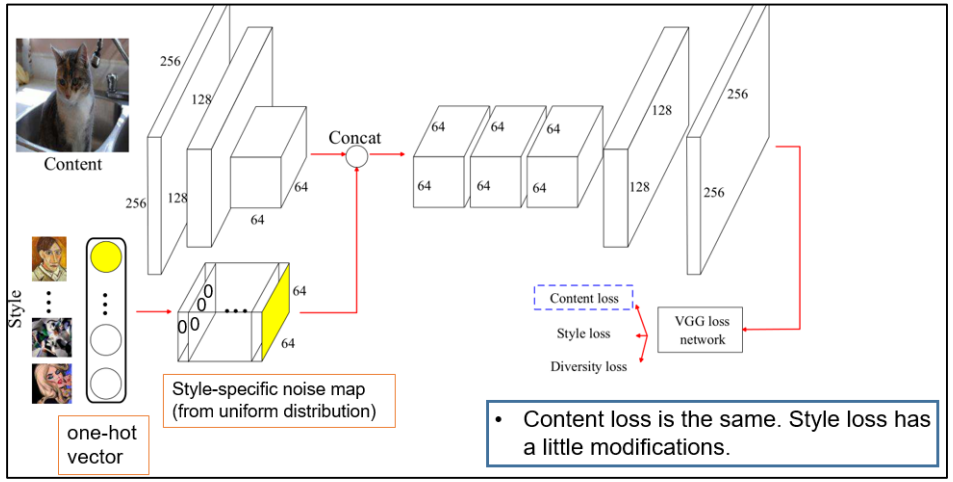
- Content loss is the same. Style loss has a little modifications.

Method



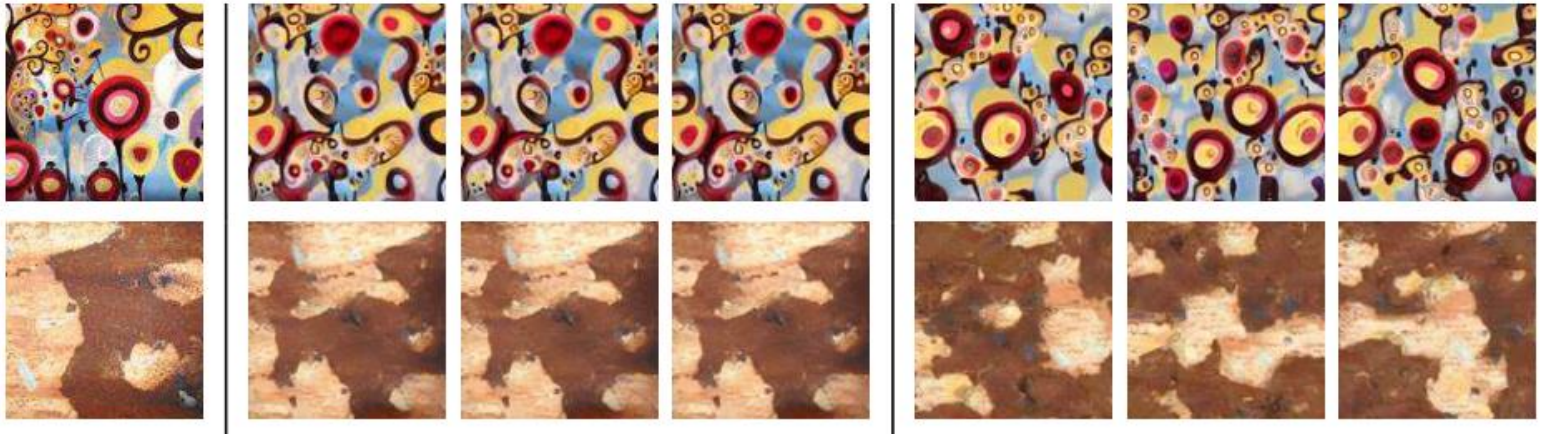
- Diversity loss
  - Penalize the **difference** of different outputs of **the same style** in the **feature space**
  - Assume the output stylized results in a batch of images are:
    - $\{P_1, P_2, \dots, P_N\}$ (they are stylized images)
    - Let $\{Q_1, Q_2, \dots, Q_N\}$ be a random reordering of $\{P_1, P_2, \dots, P_N\}$ and $P_i \neq Q_i$
    - $L_{diversity} = -\frac{1}{N}\sum_{i=1}^{N}\|\Phi(P_i) - \Phi(Q_i)\|_1$, Φ *is the feature map of con4_2 layer in VGG*

Method



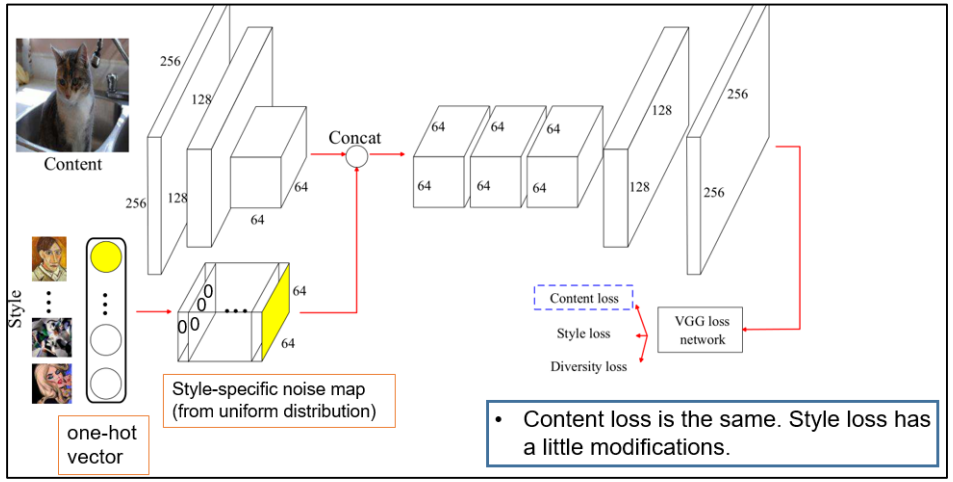
- Diversity loss
  - Penalize the **difference** of different outputs of **the same style** in the **feature space**
  - Assume the output stylized results in a batch of images are:
    - $\{P_1, P_2, \dots, P_N\}$ (they are stylized images)
    - Let $\{Q_1, Q_2, \dots, Q_N\}$ be a random reordering of $\{P_1, P_2, \dots, P_N\}$ and $P_i \neq Q_i$
    - $L_{diversity} = -\frac{1}{N}\sum_{i=1}^{N}\|\Phi(P_i) - \Phi(Q_i)\|_1$, Φ *is the feature map of con4_2 layer in VGG*

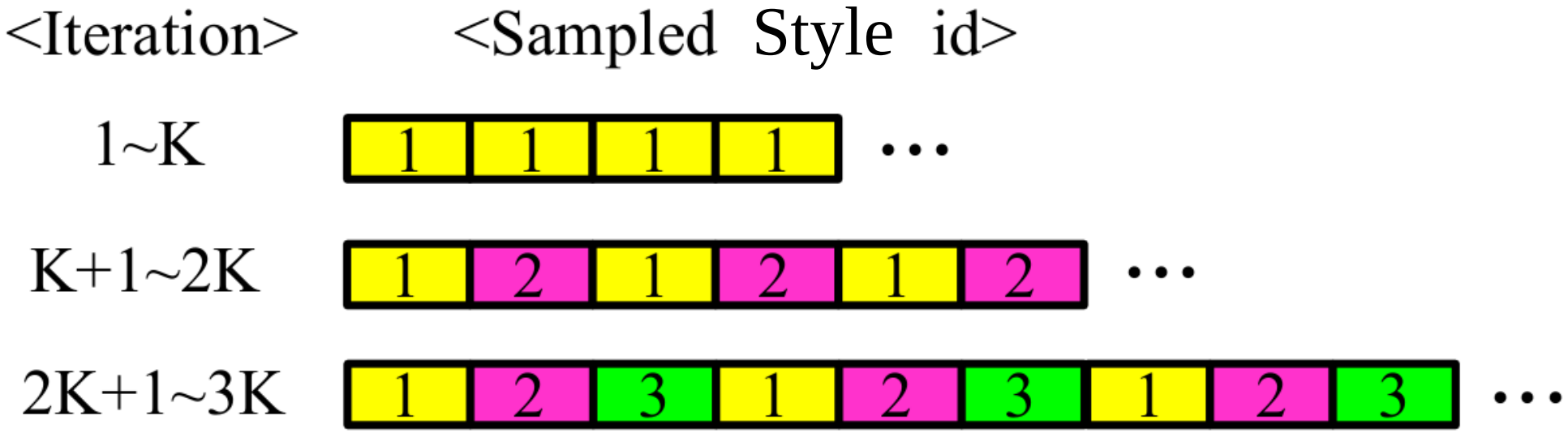
Style | Without diversity loss | With diversity loss



Method

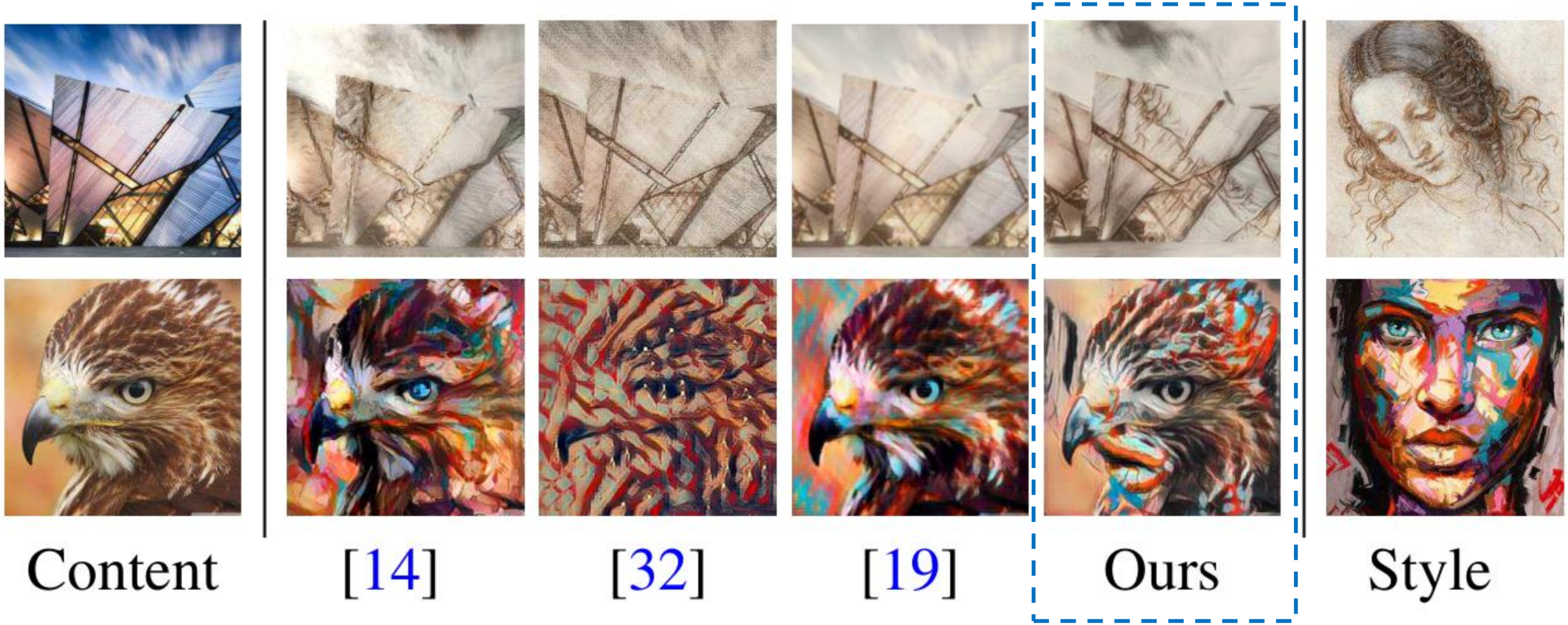
- Incremental learning:
  - Similar to curriculum learning.
  - Do not forget what is learned and learn new thing.





I doubt its training time. It is not that good as StyleBank.

Experimental Results



Conclusion

- Training strategy inspired by curriculum learning
- Diversity loss

Q & A

Question?